TECHNICAL DEEP-DIVE

AI Studio
Understanding the Full Project

A plain-English guide to every tool, concept, and engineering decision in the AI Studio codebase — from embeddings to multi-agent orchestration.

Vaibhav Mishra

FRONTEND DEV · BUILDING WITH AI

Stack	FastAPI · React 18 · ChromaDB · sentence-transformers
LLM	Ollama (local) · Groq (cloud)
Tools	10 panels · 4 retrieval strategies · 4 chunking strategies
Tests	39 pytest · SSE streaming throughout

Table of Contents

Understanding AI Studio — A Complete Guide

This document explains the entire project in plain English. Read it once and you'll be able to answer any question about what was built, why it was built that way, and how it works under the hood.

The Big Picture

What is AI Studio?

It started as a simple RAG chatbot — upload a PDF, ask questions, get answers. But the goal was bigger: build a project that demonstrates every major AI/ML engineering pattern in one place.

So it grew into ten tools. Each tool is independently useful, but together they cover the full landscape of modern AI application development: document understanding, agentic reasoning, multi-agent orchestration, visual workflow building, embedding visualization, pipeline evaluation, and more.

The key constraint: **no OpenAI, no vendor lock-in**. Everything runs locally with Ollama or switches to Groq (cloud) with a single environment variable change. Same code, same API surface.

Why does this project exist?

Two reasons:

1. **It's a real tool** — you can actually use it to chat with documents, build AI pipelines visually, generate portfolio sites from resumes, or study for a quiz on any topic.

1. **It's a portfolio showcase** — every feature maps to something interviewers ask about. RAG, ReAct, multi-agent, SSE streaming, vector search, hybrid retrieval — all of it is here, working, in production-grade code.

How does the backend talk to the frontend?

The frontend (React, running in your browser) makes HTTP requests to the FastAPI backend (Python, running on port 8000). Every request that involves an LLM uses **SSE — Server-Sent**

Events — which lets the backend push tokens one at a time as the LLM generates them. The user sees words appearing in real time instead of waiting for the full response.

All API routes require an `X-API-Key` header (`enterprise-rag-secret` by default). The backend middleware checks this on every request and rejects anything without it.

The Foundation: Embeddings and Vector Search

Before any of the tools make sense, you need to understand embeddings. Everything else builds on this.

What is an embedding?

An embedding is a way to turn text into numbers so a computer can measure how similar two pieces of text are.

Specifically: you feed a sentence into a small neural network (the embedding model — in this project it's `all-MiniLM-L6-v2` from sentence-transformers), and it outputs a list of 384 numbers. That list of numbers is the embedding.

The magic is in *what those numbers represent*. The model was trained so that sentences with similar meaning end up with similar numbers. "The cat sat on the mat" and "A feline rested on the rug" will have embeddings that are close together in 384-dimensional space. "The cat sat on the mat" and "Quarterly earnings exceeded expectations" will be far apart.

Closeness is measured with **cosine similarity** — a math formula that outputs a number between -1 and 1. Two identical sentences: ~1.0. Two completely unrelated sentences: ~0.0.

What is ChromaDB?

ChromaDB is the database that stores embeddings. Think of it as a database where instead of querying by ID or column value, you query by *similarity*.

You give it a query embedding and say "find me the 5 most similar things you have stored." It does the cosine similarity math across every stored embedding and returns the closest ones.

In this project, ChromaDB stores chunks of your uploaded documents. When you ask a question, the question gets embedded, and ChromaDB finds the document chunks most relevant to that question.

What is a collection?

In ChromaDB, a **collection** is a named group of embeddings — like a table in a SQL database. Each uploaded document creates its own collection. You can have multiple documents uploaded simultaneously and query any of them.

Tool 1: RAG Chat

What is RAG?

RAG stands for **Retrieval-Augmented Generation**. It's the technique of retrieving relevant information first, then using that information to generate an answer.

Without RAG: you ask an LLM a question and it answers from its training data. It might hallucinate (make things up), and it definitely doesn't know about your private documents.

With RAG: before the LLM answers, the system retrieves the most relevant chunks from your documents and puts them in the LLM's prompt. The LLM then answers based on *that specific context*, not its general training. No hallucination because it's told to only answer from what's provided.

The RAG pipeline step by step

1. **Upload** — you upload a PDF/Word/text file
2. **Chunk** — the document gets split into small overlapping pieces (chunks)
3. **Embed** — each chunk gets converted into a 384-number embedding
4. **Store** — embeddings + original text stored in ChromaDB
5. **Query** — you type a question; the question gets embedded
6. **Retrieve** — ChromaDB finds the top-k most similar chunks
7. **Re-rank** — a cross-encoder re-scores those chunks for precision
8. **Generate** — the LLM gets: system prompt + retrieved chunks + your question → streams an answer

Why chunking?

LLMs have a context window limit — you can't feed in an entire 300-page book. Chunking breaks the document into pieces small enough to fit. Overlap between chunks (200 characters by default) prevents information from being cut off at chunk boundaries.

The 4 chunking strategies

Fixed Size — split every N characters, with M characters of overlap. Simple. Ignores sentence and paragraph boundaries. Fast but crude.

Recursive — tries to split at paragraph breaks first, then sentence breaks, then word breaks, then characters. Respects the natural structure of the document. This is the default in LangChain and the best general-purpose choice.

Sentence Window — splits into individual sentences. But when retrieving, it returns each sentence *plus the sentences around it* for context. Good for precision — you store fine-grained units but retrieve with wider context.

Semantic — groups sentences together based on embedding similarity. Sentences that talk about the same topic end up in the same chunk, even if they're not adjacent. Most sophisticated, but slowest.

The 4 retrieval strategies

Naive Dense — embed the query, find the closest chunks by cosine similarity. Fast and simple. Good baseline. Fails when the question uses different words than the document.

Hybrid BM25 + Dense — combines two scores: BM25 (keyword matching, same algorithm as Elasticsearch) and dense similarity (embeddings). Blended with a tunable alpha. Best all-rounder — catches both exact keyword matches and semantic matches.

HyDE (Hypothetical Document Embeddings) — instead of embedding the question, ask the LLM to write a hypothetical answer first, then embed that hypothetical answer. The idea: a hypothetical answer looks more like a document passage than a question does, so it retrieves better. Great for vague queries.

Multi-Query + RRF — generate 3 differently-phrased versions of the question, retrieve top-k for each phrasing, then merge the three result lists using Reciprocal Rank Fusion. Best recall — the same chunk might rank poorly for one phrasing but highly for another. Costs 4× the API calls.

What is cross-encoder re-ranking?

After retrieving 20 candidate chunks (using the fast methods above), a **cross-encoder** re-scores them more carefully.

The difference: the retrieval embeddings encode the question and each chunk *separately*, then compare. Fast but approximate. The cross-encoder reads the question and chunk *together*, as a pair,

and outputs a single relevance score. Much more accurate because it can consider interactions between the words, but too slow to use on all chunks in the database.

Two-stage retrieval: fast bi-encoder gets candidates → slow cross-encoder picks the best. Best of both worlds.

Tool 2: ReAct Agent

What is an agent?

A regular LLM call is one shot — question in, answer out. An agent is a loop. The LLM can take actions, observe results, and use those results to decide what to do next.

What is the ReAct pattern?

ReAct = **Re**asoning + **Act**ing. The LLM is prompted to output its thinking in a structured format:

```
Thought: I need to find the population of France.
Action: web_search
Action Input: population of France 2024
Observation: France has a population of approximately 68 million.
Thought: I have the answer.
Final Answer: France's population is approximately 68 million.
```

The backend parses this output with regex, detects the `Action:` line, executes the tool, feeds the result back as `Observation:`, and loops again. This continues up to 7 iterations (MAX_ITERATIONS guard).

What tools does the agent have?

- **ChromaDB** — search your uploaded documents
- **DuckDuckGo** — web search (no API key needed, uses the DuckDuckGo public search)
- **Calculator** — evaluate math expressions safely
- **Direct Answer** — when the LLM decides no tool is needed

Why ReAct over simple RAG?

RAG is one retrieval → one answer. ReAct can: search docs → read result → search web → combine → calculate → answer. It handles multi-step questions that require connecting information from multiple sources.

The limitation: it's slow (multiple LLM calls) and regex parsing is brittle. Structured tool calling (function calling) would be more reliable — it's on the roadmap.

Tool 3: Multi-Agent Pipeline

What is multi-agent?

Multi-agent takes the idea of specialized roles further. Instead of one LLM trying to do everything, you chain multiple LLM calls where each has a specific job and sees what the previous ones produced.

Example — Research → Write → Review:

1. **Research Agent** — searches web/docs, compiles findings
2. **Writer Agent** — takes Research output, writes a structured response
3. **Reviewer Agent** — takes Writer output, critiques it, suggests improvements

Each agent sees: its own system prompt (defining its role) + the original user input + every previous agent's full output.

Why is this better than one big prompt?

Each agent can be focused. A researcher doesn't need to worry about prose quality. A writer doesn't need to evaluate factual accuracy. A reviewer only needs to critique. Focused prompts produce more reliable outputs than "do everything" prompts.

It also mirrors how teams actually work — specialization beats generalization for complex tasks.

The 3 built-in templates

Research → Write → Review — general-purpose content production pipeline. Best for anything where quality and accuracy both matter.

Analyze → Summarize → Format — deep analysis first, then distill to key points, then format for the target audience. Good for long documents or complex data.

Search → Synthesize → Answer — web/doc search, then synthesize findings across sources, then write a final answer. Good for research questions.

How does streaming work here?

Each agent streams tokens live. The frontend shows a panel per agent, and tokens appear in real time as each agent writes. When one agent finishes, the next one starts. SSE event types: `pipeline_start` → `agent_start` → `agent_tool` → `agent_token` → `agent_done` → `done`.

Tool 4: Agent Workflow Builder

What is it?

A visual drag-and-drop interface to build AI pipelines without writing code. You add nodes, connect them with edges, and hit Run. The backend executes them in the right order.

What is a DAG?

DAG = Directed Acyclic Graph. "Directed" means connections have a direction (A feeds into B, not B into A). "Acyclic" means no loops (A → B → C → A would be a cycle and is illegal).

A DAG is the right structure for a pipeline because it guarantees there's a valid execution order — some nodes must finish before others can start.

How does execution order work? (Kahn's algorithm)

Topological sort — find the order to execute nodes so every node runs only after all its inputs are ready.

Kahn's algorithm:

1. Find all nodes with no incoming edges (start nodes). Add them to a queue.
2. Process a node from the queue. Reduce the "incoming edge count" of all its neighbors.
3. Any neighbor whose incoming count hits 0 gets added to the queue.
4. Repeat until the queue is empty.

Result: a valid execution order. If there's a cycle, the algorithm can't complete — that's how you detect illegal pipelines.

The 6 node types

- **Input** — the user's text goes in here. Starting point.
- **LLM Call** — sends a prompt to the LLM, streams a response. Supports `{{nodeId}}` to inject upstream outputs into the prompt.
- **Retrieval** — queries ChromaDB with a search term, returns relevant chunks.
- **Web Search** — DuckDuckGo search, returns results.
- **Transform** — Python-like text operations: uppercase, lowercase, extract first N chars, trim.
- **Output** — displays the final result. End point.

What is `{{nodeId}}` injection?

When you write a prompt for an LLM node, you can reference any upstream node's output using its ID in double curly braces: `{{node_1}}`. Before the LLM call, the engine replaces `{{node_1}}` with the actual text output of that node. This is how you pass information between nodes.

Tool 5: Visualize

Three sub-tools that make invisible pipeline behavior visible.

Embedding Space (PCA scatter plot)

Every chunk in your ChromaDB collection gets fetched with its embedding (384 numbers). Those 384 dimensions get compressed to 2 dimensions using PCA (Principal Component Analysis). Each chunk becomes a dot on a 2D scatter plot.

Why is this useful? You can visually see which chunks are semantically similar (clustered together) vs unrelated (spread apart). You can type a query and see where it lands relative to your document chunks — this reveals whether your retrieval strategy will find the right chunks.

What is PCA? A math technique that finds the two directions of maximum variance in high-dimensional data and projects everything onto those two directions. It's lossy (you lose most of the information) but preserves the most important structure. Think of it as "the shadow" cast by 384-dimensional data onto a 2D wall.

Context Window Inspector

Before the LLM generates an answer, its prompt contains: system message + retrieved chunks (each with relevance score) + your question. This tool shows you exactly that prompt, with token counts per section and a color-coded bar showing how full the context window is.

This answers: "Is the LLM even seeing the right information?" If retrieved chunks are irrelevant, this is where you'll catch it.

Chunking Visualizer

Paste any text and see all 4 chunking strategies applied simultaneously. For each strategy: how many chunks it produces, average chunk size, and the actual chunk boundaries highlighted in the text. Lets you compare strategies side-by-side before uploading a document.

Tool 6: RAG Evaluation

Why evaluate?

"The chatbot seems to work" is not good enough in production. You need numbers. RAG evaluation gives you quantitative metrics so you can compare retrieval strategies, tune chunk sizes, and catch regressions.

The 3 metrics

Context Relevance — after retrieving chunks for a question, how similar are those chunks to the question? Measured with cosine similarity between the question embedding and each retrieved chunk's embedding. A low score means retrieval is failing — wrong chunks are being fetched.

Answer Faithfulness — does the generated answer actually follow from the retrieved context? An LLM judge reads the answer and the context and scores whether the answer is supported by the context or is making things up. High faithfulness = no hallucination.

Answer Relevance — how similar is the generated answer to the original question? Measured by embedding similarity. A high score means the answer actually addresses what was asked. A low score means the answer went off-topic.

How to use it

Paste a list of questions (one per line), pick a retrieval strategy, hit Run. For each question, you get all 3 scores. Aggregate bars at the bottom show overall pipeline quality. Use it to compare: does Hybrid retrieval produce higher context relevance than Naive Dense on your document?

Tool 7: Quiz Game

The LLM generates multiple-choice questions about any topic, optionally grounded in your uploaded documents. Pick a topic → the model suggests subtopics → you pick which ones → questions get generated → you answer them timed → full post-game analysis.

Questions can come from three sources: your uploaded documents (RAG-grounded), web search (current information), or the LLM's own knowledge. Scored and timed. The post-game analysis shows which questions you got wrong and why.

Tool 8: CV → Portfolio

Upload a PDF resume. The backend extracts the text, sends it to the LLM with a structured prompt asking it to parse the resume into JSON (name, experience, skills, education, etc.). That JSON is returned to the frontend, which renders it into one of 5 animated portfolio templates.

Templates: Basic (clean gradient), Creative (dark with animated orbs), Dark (terminal/CRT aesthetic), Old School (parchment letterhead), 90s (GeoCities chaos with Windows 95 dialogs).

Tool 9: Blueprint

An interactive documentation page built into the app. Eight sections: setup guide, architecture diagrams, retrieval strategy comparisons, chunking strategy comparisons, concept explainers, system metrics (live), performance analytics, and 14 interview Q&As with model answers.

If someone asks you an AI/ML engineering interview question about anything in this project, the Blueprint tab has a pre-written answer.

The Infrastructure

SSE Streaming — how real-time output works

When you make a request to any streaming endpoint (chat, agent, multi-agent, workflow, eval, quiz), the server doesn't wait until it has the full answer to respond. Instead, it opens a persistent HTTP connection and sends data in chunks as it's generated.

The wire format is simple:

```
data: {"type": "token", "content": "Hello"}\n\n

data: {"type": "token", "content": " world"}\n\n

data: {"type": "done"}\n\n
```

The frontend reads this stream with a `ReadableStream` and processes each JSON event as it arrives. This is why you see tokens appearing one by one — each token is a separate SSE event.

Why SSE and not WebSockets? SSE is simpler — it's one-directional (server → client), works over regular HTTP, and doesn't need a special protocol upgrade. For LLM token streaming where the client only sends one request and receives a stream, SSE is the right choice.

ProcessContext — the real-time terminal sidebar

Every tool in the app logs events to a shared context. The terminal sidebar (bottom-right) shows these events in real time with color-coding by type.

It works like a pub/sub event bus using React's Context API:

- `useProcess()` hook exposes a `log(tag, message, status)` function
- Any component anywhere in the app can call `log()` without knowing about any other component
- The TerminalSidebar subscribes to the context and renders new events as they arrive

Tags: SYSTEM (gray), QUERY (blue), EMBED (purple), RETRIEVAL (indigo), CONTEXT (cyan), MODEL (green), STREAM (green), AGENT (violet), TOOL (orange), WORKFLOW (rose), MULTIAGENT (pink), VISUALIZE (indigo), etc.

Rate Limiting

To prevent abuse on the live demo, each user gets 100 LLM-heavy requests per day. The user is identified by a UUID stored in `localStorage` and sent as the `X-User-ID` header on every request.

The backend stores request counts in SQLite (`rate_limiter.db`). Each row: user UUID, request count, date. At the start of each request, it checks: if today's count >= 100, reject. Otherwise, increment and allow. The date comparison handles the midnight reset automatically — a new date means a fresh row.

SQLite is used (not Redis) because this is a single-instance app. If you needed to rate-limit across multiple backend instances, you'd need Redis.

Authentication

A single shared secret (`enterprise-rag-secret` by default, configurable in `.env`). The FastAPI middleware reads the `X-API-Key` header on every request. If it's missing or wrong, the request gets a 403 before it ever reaches a route handler.

This is a simple approach appropriate for a single-user local app or demo. Production would use JWTs or OAuth.

Testing

39 pytest tests that run in ~2 seconds with no model downloads. They test:

- Chunking strategies produce the right number of chunks with the right overlap
- Retrieval helper functions correctly format ChromaDB results
- API endpoints return the right status codes and response shapes (with ChromaDB and Ollama mocked)

The tests don't test actual LLM quality — that's what the RAG Evaluation tool is for.

How the Frontend is Structured

The sidebar navigation

The left sidebar has a tab for each tool. It can collapse from 200px (icon + label) to 60px (icon only). Each tab has its own accent color — chat is brand-blue, agent is violet, multi-agent is pink, etc. On mobile, the sidebar turns into a scrollable tab bar at the bottom of the screen.

Mobile responsiveness

- **MultiAgentPage** — uses a tab bar at the top (Pipeline config / Output) to switch between the left panel and right panel. On desktop, both panels show side by side.
- **WorkflowPage** — on mobile, hides the NodePalette and ConfigPanel (they need precise mouse interaction). The React Flow canvas itself supports touch, so you can view and interact with the pipeline on mobile.
- **VisualizePage** — the 3-tab grid collapses to single column on small screens.
- Everything else — responsive by default using Tailwind's `sm:` and `md:` breakpoints.

State management philosophy

No Redux. No Zustand. Just:

- `useState` / `useReducer` for local component state
- `useContext` for shared state that multiple components need (process logs, active collection)
- URL/query params for nothing — navigation is handled in `App.tsx` with a `activeTab` state variable

This is intentional. For this app's complexity, React's built-in tools are sufficient. Adding a global state library would be premature.

The LLM Layer

How Ollama and Groq both work

Both are configured in `services/llm.py`. Two functions: `stream_answer()` (for RAG chat — builds the full messages list from retrieved context) and `stream_messages()` (generic — takes an arbitrary messages list, used by multi-agent and agent).

Both functions check `settings.llm_provider`:

- `"ollama"` → uses the `ollama` Python library, connects to `localhost:11434`
- `"groq"` → uses the `groq` Python library, connects to Groq's cloud API

The caller code never needs to know which one is active. Switching providers is one line in `.env`.

Why Groq?

Groq runs LLMs on custom LPU (Language Processing Unit) hardware that's extremely fast — typically 10x faster token generation than GPU inference. They have a free tier. It's the best option for running this project in the cloud without a GPU or without paying OpenAI.

Common Questions and How to Answer Them

"Why not just use LangChain?"

LangChain abstracts over all of this. Building it from scratch means understanding exactly what's happening at each step. You can't debug a LangChain RAG pipeline if you don't understand what retrieval, chunking, and re-ranking actually do. This project builds those same primitives manually, which means you can explain every line.

"Why ChromaDB and not Pinecone/Weaviate/pgvector?"

ChromaDB runs in-process with no external service needed — perfect for local development and demos. The concepts (collections, embeddings, cosine similarity search) are identical across all vector databases. Switching to Pinecone or pgvector would be a service layer change, not a conceptual change.

"How is multi-agent different from LangGraph or CrewAI?"

LangGraph and CrewAI are frameworks that handle the orchestration for you. This project builds the orchestration from scratch — sequential loop, context passing, tool dispatch — so you can see exactly what "multi-agent" means at the code level. Same pattern, no magic.

"What's the difference between your ReAct agent and a simple chain?"

A chain is fixed — step 1 always leads to step 2. An agent is dynamic — the LLM decides at each step which tool to use (or whether to stop). The same agent handles a simple one-step question and a complex five-step question without you changing anything.

"Why SSE instead of WebSockets?"

SSE is one-directional (server pushes to client). WebSockets are bidirectional. For LLM token streaming, the client sends one request and then just listens — bidirectionality isn't needed. SSE is simpler to implement, works over standard HTTP, and doesn't require connection upgrade negotiation.

"How does HyDE actually help?"

Normal retrieval: embed "What causes inflation?" → find chunks similar to that question. Problem: chunks in the database are statements, not questions. They don't look like questions in embedding space. HyDE: ask the LLM to write a hypothetical answer like "Inflation is caused by..." → embed that → find chunks similar to that answer. The hypothetical answer looks like the real document passages, so retrieval is more accurate.

"What is Reciprocal Rank Fusion?"

When you retrieve from 3 different query phrasings, each returns a ranked list. You can't just average scores (they might be on different scales). RRF formula: for each document, sum `1 / (rank + 60)` across all lists where it appears. This rewards documents that appear near the top of multiple lists, regardless of their absolute score. Robust, simple, and surprisingly effective.

"Why 60 in the RRF formula?"

It's a smoothing constant. Without it, a rank-1 result gets score 1.0, rank-2 gets 0.5 — too dramatic a drop. With 60: rank-1 gets 1/61 ≈ 0.016, rank-2 gets 1/62 ≈ 0.016 — much smoother, prevents one very high-ranked result from drowning out everything else. The value 60 was established empirically in the original RRF paper and has held up in practice.

"What would you change if this were a production system?"

- Replace `X-API-Key` shared secret with JWT auth
- Replace SQLite rate limiter with Redis (needed for multi-instance deployments)
- Replace regex-parsed ReAct with structured function calling (more reliable)
- Add proper logging (structured JSON logs, not just the terminal sidebar)
- Add monitoring (Prometheus metrics, error tracking)
- Add a proper job queue for long-running eval runs (Celery or similar)
- Add document metadata storage in a real database (not just ChromaDB)

Mental Model: How a Request Flows End to End

Here's what actually happens when you type a question in RAG Chat and hit send:

1. **Frontend** — React calls `streamChat(question, collection, strategy)` in `api/client.ts`
2. **HTTP** — `POST /api/v1/chat` with `X-API-Key` header
3. **Middleware** — FastAPI auth middleware validates the API key
4. **Rate Limiter** — checks SQLite: has this UUID made <100 requests today?

5. **Route handler** — `chat.py` receives the request, calls the retrieval service
6. **Embed query** — `all-MiniLM-L6-v2` converts the question to 384 numbers (5ms)
7. **Retrieve** — ChromaDB returns top-20 chunks by cosine similarity
8. **Re-rank** — cross-encoder re-scores, returns top-5
9. **Build prompt** — system prompt + top-5 chunks + question assembled into messages list
10. **LLM stream** — Ollama or Groq starts generating tokens
11. **SSE** — each token is wrapped in `data: {...}\n\n` and flushed to the HTTP response
12. **Frontend** — `ReadableStream` decodes each event, calls `setMessages()` to append the token
13. **React renders** — the new character appears in the chat bubble
14. **Process monitor** — each step (EMBED, RETRIEVAL, CONTEXT, STREAM) logged to ProcessContext
15. **Done event** — `{"type": "done"}` closes the stream, frontend stops the loading state
16. **Performance log** — total retrieval time + LLM time saved to SQLite `perf.db`

That's the full round trip. Every other tool follows the same pattern with variations on steps 6–11.

Things Worth Memorizing

- **Embedding model:** `all-MiniLM-L6-v2` — 384 dimensions, 80MB, ~5ms per query
- **Re-ranker:** `ms-marco-MiniLM-L6-v2` — cross-encoder, reads query+chunk together
- **ReAct iterations:** MAX 7 — prevents infinite loops
- **Rate limit:** 100 requests/day per UUID — SQLite, auto-resets at midnight
- **Default chunk size:** 1000 characters, 200 overlap
- **Context window:** shown in the Context Window Inspector tool
- **Vector DB:** ChromaDB — in-process, no external service needed
- **SSE format:** `data: {json}\n\n` — two newlines end each event
- **DAG algorithm:** Kahn's topological sort — finds valid execution order
- **RRF constant:** 60 — smoothing factor in `1/(rank+60)`
- **Tests:** 39 pytest tests, ~2 seconds, no model downloads needed
- **LLM providers:** Ollama (local) or Groq (cloud) — same API surface, one env var to switch